

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter describes the methodology adopted for the design and implementation of a cloud-based healthcare data management system for University of Ilorin clinic using AWS. It explains the research design, system development approach, data collection techniques, system architecture, implementation tools, and evaluation methods. The methodology was carefully selected to ensure that the resulting system effectively addresses identified challenges in healthcare data management system in University of Ilorin clinic while remaining practical within the institutional and infrastructural context of a Nigerian university.

3.2 Research Design

This study adopted a system development research design with a quantitative requirement gathering approach. The research focuses on the design and implementation of a cloud-based healthcare data management system for the University of Ilorin Clinic using Amazon Web Services (AWS). Data required for system development were obtained through questionnaires, interviews, and observations conducted among selected stakeholders within the clinic.

3.2.1 Methodology

The methodology adopted for developing the cloud-based healthcare data management system is the Agile Scrum framework. Agile Scrum is an iterative and incremental approach that allows for flexibility, rapid adaptation, and continuous feedback throughout the development lifecycle. This model consists of structured cycles known as "Sprints," which ensure that the system evolves through a series of manageable increments. For this project, the Scrum methodology provides a collaborative way to manage requirements through a dynamic Product Backlog. The process began with backlog refinement, where system needs such as secure data storage, patient record management, and remote accessibility were translated into User Stories. These stories represent the requirements of various stakeholders, including clinical staff and administrators, focusing on practical outcomes like real-time data synchronization and HIPAA compliant access controls.

During the Sprint Planning phase, the development team identifies high-priority user stories to be tackled in the upcoming cycle. With an iterative design approach, where system architecture and database models for patient records and billing are refined iteratively this allows the project to remain responsive to technical challenges or shifting regulatory requirements in the healthcare landscape.

The implementation phase is integrated into the sprint cycles, where the system is built using a modern cloud native tech stack. The frontend is developed using React.js to ensure a responsive user experience across devices, while the backend utilizes Node.js and

Express to manage API communication and healthcare logic. AWS DynamoDB serves as the NoSQL database, providing the schema flexibility required to handle diverse healthcare data types, from structured reports to semi-structured clinical notes.

Testing is not a separate end-phase but is conducted continuously within each sprint. Unit testing and integration testing are performed on every product increment to catch bugs early and prevent error propagation. At the end of each sprint, a Sprint Review is held to demonstrate the functional increment, ensuring the software meets real-world clinical expectations before moving to the next cycle.

Deployment and maintenance follow a Continuous Integration and Continuous Deployment (CI/CD) pipeline. Each successful sprint results in a potentially shippable product increment hosted on cloud infrastructure, making the system accessible via web browsers while maintaining high availability. This iterative deployment ensures that critical features reach users faster, reducing the "know-do gap" often found in traditional healthcare IT projects.

The Agile Scrum framework was chosen for its responsiveness to changing requirements by 40% in healthcare settings . It is particularly suitable for cloud-based healthcare systems where data security, interoperability, and the integration of emerging technologies like AI require an adaptable and transparent development path.

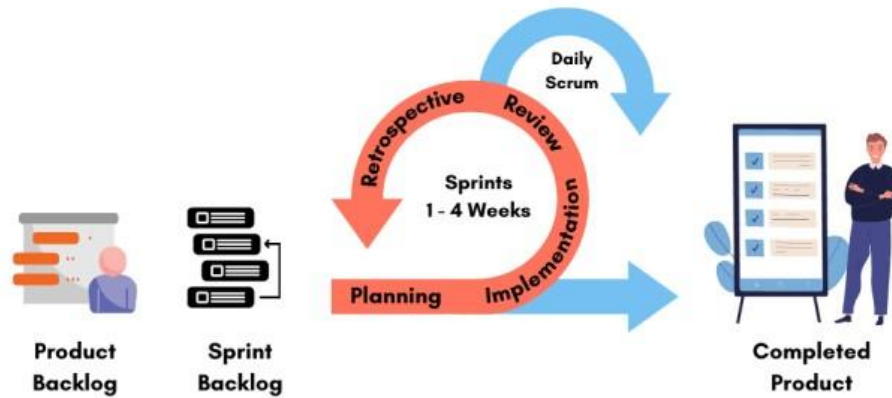


Figure 1: Agile Scrum Framework

3.2.1.1 Agile Scrum Sprints Backlogs

Sprint	Objectives	Key Activities	AWS Services Used
Sprint 0	Project initialization	Stakeholder consultation, sprint backlog creation, AWS environment setup, IAM role configuration, DynamoDB schema planning	IAM, DynamoDB
Sprint 1	Authentication & Role Management	Configure Cognito user pool, implement JWT authentication, define RBAC (Admin, Doctor, Nurse, Receptionist)	Cognito, IAM
Sprint 2	Patient Records Module	Create DynamoDB tables, implement CRUD operations, integrate backend APIs	DynamoDB, EC2/Lambda, API Gateway

Sprint 3	Appointment & Notification Module	Implement appointment booking, integrate SES for email confirmations, configure SNS for admin alerts	DynamoDB, SNS
Sprint 4	Security Hardening	Enable HTTPS, configure IAM least privilege policies, enable MFA, integrate KMS encryption	Cognito, IAM, KMS
Sprint 5	Monitoring, Backup & Alerts	Enable DynamoDB backups, configure CloudWatch alarms, connect alarms to SNS notifications	DynamoDB, CloudWatch, SNS

Table 1: Agile scrum Sprints Backlog

3.2.2 Population and Location of Requirement Gathering

This section provides an overview of the target population and the location of the requirements gathering. This was conducted at the University of Ilorin Clinic, located within the University of Ilorin, Kwara State, Nigeria. The target population consisted of stakeholders who interact with patient records and healthcare information within the clinic. These include:

S/N	STAKEHOLDER	POPULATION
1	Director	1
2	Receptionist	4
3	Doctors	5
4	Nurses	10
5	Pharmacists	3

6	Students (Patients)	46712
Total		46735

3.2.3 Sampling Technique

The sample for the requirement gathering was selected using Purposive Sampling. This sampling technique involves deliberately selecting individuals who possess relevant knowledge, experience, or responsibilities related to the research objectives. Purposive sampling was adopted due to the system development nature of this project, The respondents were selected based on their roles within the clinic, including the Director, Doctors, Nurses, Receptionists, Pharmacists, and a small number of Students who utilize the clinic's services. This approach ensured that relevant information was obtained from participants who are directly involved in the creation, management, and use of healthcare records.

STAKEHOLDER	POPULATION	SAMPLE SELECTED
Director	1	1
Receptionist	4	1
Doctors	5	1
Nurses	10	1
Pharmacists	3	1
Students (Patients)	46712	10

3.2.4 Data Collection Methods

The primary data collection method for this study was survey questionnaires, which was administered physically. These questionnaires provided an efficient and structured way to gather data. The survey consists of closed-ended questions, offering predefined response options that enabled quantitative analysis. This structured approach ensured the collection of large scale, measurable data from a diverse group of participants.

3.2.5 Ethical Considerations

Ethical principles were observed throughout this study. Permission was obtained from the University of Ilorin Clinic before data collection. Participation was voluntary, and respondents were informed of the purpose of the study. No patient medical records were collected, and all information obtained was used strictly for academic purposes. The confidentiality and anonymity of all respondents were maintained throughout the research process.

3.3 Analysis of the Existing System

The University of Ilorin clinic provides comprehensive healthcare to students and staffs and the existing data management process at there is still largely manual and paper based. Patient records are primarily stored in physical files, this approach limits efficiency, accessibility, and data security. Record retrieval depends on manual searching through paper files, which can be time-consuming, especially during peak clinic hours. Updates to patient information require physical file handling, increasing the chances of misplaced or damaged records.

The current system also lacks automated backup mechanisms. Data stored locally is vulnerable to physical damage, accidental loss, or system failure. Furthermore, access control is mostly procedural rather than technical, meaning sensitive patient information may not be adequately protected through systematic authentication mechanisms.

3.3.1 Drawbacks of the Existing System

The major limitations of the traditional data management system used at the clinic include:

- i. Slow and inefficient retrieval of patient records
- ii. Risk of data loss due to physical damage
- iii. Limited accessibility and lack of centralized data storage
- iv. Weak technical access control mechanisms
- v. No automated backup or disaster recovery process
- vi. Poor scalability as patient population increases

3.3.2 Analysis of the Proposed System

The proposed system is a cloud-based healthcare data management system hosted on AWS. It replaces manual record keeping with a centralized digital infrastructure that securely stores and manages patient data. The system enables authorized users to access patient records through secure authentication mechanisms powered by Amazon Cognito. Role-based access control ensures that users can only access data relevant to their responsibilities. The cloud infrastructure ensures high availability, scalability, and automated backup, addressing the operational limitations of the existing system.

3.3.2 Benefits of the Proposed System

The proposed system that has been analysed above will eliminate the drawbacks of the existing system. Its benefits include:

- i. Faster patient data retrieval and improved workflow efficiency
- ii. Centralized and secure data storage
- iii. Role-based access control for enhanced security
- iv. Automated backups and disaster recovery using AWS services
- v. Scalability to accommodate increasing patient records
- vi. Real-time access to patient information across authorized users
- vii. Improved monitoring and system reliability

3.4 Requirement Specification:

The requirement specification is divided into Functional and Non-Functional requirements.

3.4.1 Functional Requirements

3.4.1.1 Student Registration and Queue Management Flow - Receptionist Portal

ID	Requirement Title	Description
FR-1	New Student Registration	The system MUST allow the Receptionist to register new students by capturing essential demographic and matriculation data.
FR-2	Student Record Search	The system MUST allow the Receptionist to efficiently search for existing student records using the Matriculation Number or other unique identifier.
FR-3	Record Maintenance	The system MUST allow the Receptionist to update and, with appropriate logging, delete student records, ensuring data accuracy.

FR-4	Doctor Assignment and Queuing	The system MUST allow the Receptionist to assign a student to an available Doctor's queue.
FR-5	Real-Time Queue Visualization	The system MUST display a real-time, consolidated view of the waiting queue, showing students' names and assigned doctors.
FR-6	Patient Entry Notification	The system MUST receive and display a realtime notification from a Doctor when a patient is called, signaling the Receptionist to allow patient entry.

3.4.1.2 Patient Consultation and Prescription Flow - Doctor Portal

ID	Requirement Title	Description
FR-7	Assigned Queue Access	The Doctor Portal MUST display the specific queue of students assigned to that Doctor for consultation.
FR-8	Patient Call Initiation & Notification	The Doctor MUST be able to initiate a " Call Next Patient " action, which automatically notifies both the Student Portal and the Receptionist Portal .
FR-9	Clinical File Access	The Doctor MUST be able to view the full, historical clinical file, including diagnosis history and previous prescriptions, for the active patient.
FR-10	Consultation Record & Diagnosis Entry	The Doctor MUST be able to write and save the current diagnosis and consultation notes to the student's clinical file.
FR-11	Prescription Generation & Submission	The Doctor MUST be able to generate and submit a formal prescription, including medication details, dosage, and frequency.
FR-12	Pharmacy & Student Prescription Notification	Upon submitting a prescription, the system MUST automatically notify both the Pharmacy Portal and the Student Portal that the prescription is ready for collection.

Table 3: Patient Consultation and Prescription Flow - Doctor Portal

3.4.1.3 Prescription Fulfillment Flow - Pharmacy Portal

ID	Requirement Title	Description
FR-13	New Prescription Alert	The Pharmacy Portal MUST receive and display a real-time notification when a new prescription is submitted by a Doctor.

FR-14	Prescription Lookup by Identifier	The Pharmacist MUST be able to securely look up a student's pending prescriptions using the student's Matriculation Number .
FR-15	Fulfillment Status Update	For each prescription, the Pharmacist MUST be able to mark the status as either " Collected " or " Rejected " (e.g., due to stock unavailability).
FR-16	Rejection Follow-up Notification	If a prescription is marked "Rejected," the system SHOULD automatically notify the prescribing Doctor for alternative prescription follow-up.
FR-17	Prescription Audit Listing	The Pharmacist MUST be able to pull up lists of prescriptions marked as " Given " and " Non-Given " (pending/rejected) for inventory and auditing purposes.

Table 4: Prescription Fulfillment Flow - Pharmacy Portal

3.4.1.4 Patient Information Access Flow - Student Portal

ID	Requirement Title	Description
FR-18	Secure Data Access	The student MUST be able to securely view their personal demographic and historical clinical data.

Table 5: Patient Information Access Flow - Student Portal

3.4.1.5 API & Identity Management Integration

ID	Requirement Title	Description
FR-19	Central API Endpoint	All four portals MUST connect to a single, secured Amazon API Gateway endpoint for all backend interactions.
FR-20	Identity and Authorization	Amazon Cognito MUST be used to authenticate all API requests, validating the user's identity and assigning appropriate access based on their defined role (e.g., Doctor, Pharmacist).

FR-21	Content Delivery Network (CDN)	Amazon S3 (for storage) and Amazon CloudFront (for delivery) MUST be used to host and rapidly deliver all static assets for the web portals.
--------------	---------------------------------------	---

Table 6:API & Identity Management Integration

3.4.2 Non-Functional Requirements

The following are the non-functional requirements of the proposed system:

- i. Latency: System must respond to standard data queries in < 2 seconds
- ii. Availability: The system must achieve at least 99.9% uptime and shall be accessible 24/7, except during planned maintenance
- iii. Security: Full encryption for data at rest (AES-256) and in transit (TLS 1.2+) for protection of user data from unauthorized access or breaches
- iv. Scalability: The system can grow and handle increased demand without performance issues by allowing the addition of more servers to handle high traffic.
- v. Reliability: Data must be consistent across all portals to avoid fragmentation.

3.5 System Design

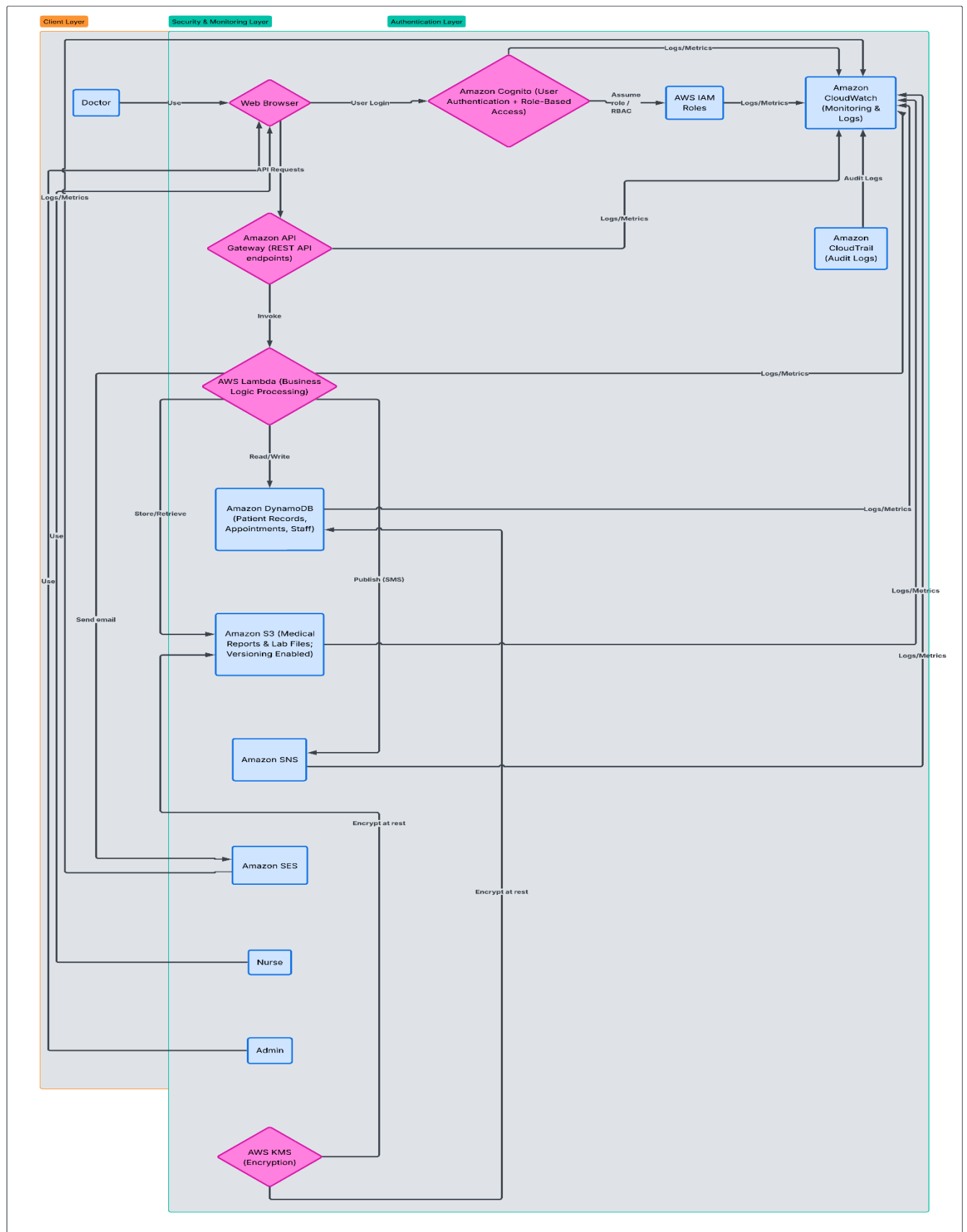


Figure 2: Cloud Architectural Design

The architectural design of the healthcare data management system illustrates how the web application, authentication services, backend processing layer, and centralized clinical database integrate to support secure patient record management, appointment scheduling, and controlled access to medical information within the University of Ilorin Clinic. The system is deployed on AWS and it follows a three-tier architecture consisting of a presentation layer, application layer, and data layer, with centralized identity and access management.

Clinic staff, the Doctors, Nurses, Pharmacist, and Administrators will access the system through a web browser over HTTPS. The frontend application will be hosted on an Amazon EC2 instance, which serves the web interface and manages user interactions.

User authentication and role-based authorization will be handled using Amazon Cognito. Cognito will authenticate users and issue secure tokens used to access protected backend resources.

The application layer will consist of Amazon API Gateway and AWS Lambda. API Gateway will securely expose RESTful endpoints and validate Cognito tokens before forwarding requests. AWS Lambda functions will implement core business logic, including patient registration, appointment scheduling, record updates, and notification triggers.

Clinical data will be stored in Amazon DynamoDB, a managed NoSQL database optimized for scalability and low-latency performance. Supporting documents such as reports and prescriptions will be stored in Amazon S3.

Notification services will be implemented using Amazon SNS and Amazon SES.

System monitoring and logging will be performed using Amazon CloudWatch and AWS CloudTrail.

Backend components will operate within a Virtual Private Cloud (VPC) to enhance isolation, while public access will be limited to the EC2-hosted frontend and API Gateway endpoints secured via HTTPS and appropriate security groups.

3.5.1 Data Flow

User requests such as patient registration, appointment booking, diagnosis entry, and prescription updates will originate from the EC2-hosted web application and be transmitted securely to Amazon API Gateway.

API Gateway will authenticate requests using Cognito tokens and route them to appropriate AWS Lambda functions. Lambda functions will validate inputs, enforce business rules, and perform CRUD operations on DynamoDB tables structured for patients, appointments, encounters, prescriptions, and audit logs.

Clinical documents uploaded by users will be stored in Amazon S3, with corresponding metadata references maintained in DynamoDB to ensure linkage between structured records and associated files.

Event-driven processes such as appointment confirmations or reminders will trigger Lambda functions to invoke Amazon SNS or Amazon SES for SMS and email notifications.

All interactions will generate logs sent to Amazon CloudWatch, while API level activity and user actions will be recorded in AWS CloudTrail to ensure traceability and accountability.

3.6 Programming Languages and Development Tools

3.6.1 Frontend Development

The frontend development of the Healthcare Data Management System was aimed at delivering a modern, responsive, and accessible interface that simplifies the process of managing patient records and scheduling appointments. It involved the use of modern web technologies to ensure a smooth user experience across devices.

The following technologies were used:

HTML5 (Hypertext Markup Language version 5): Served as the structural backbone of the web pages. It was used to organize the layout and define content such as patient registration forms, appointment dashboards, search inputs, and navigation links. Semantic HTML5 elements were also adopted to improve accessibility and clarity of the codebase.

CSS (Cascading Style Sheets): CSS was used to create an appealing visual design for the platform. Through layout grids, color themes, font styling, and responsive media queries,

CSS made the user interface visually consistent and adaptive across different screen sizes. This was crucial for enhancing the usability of the platform.

JavaScript and React.js: JavaScript powered the interactivity of the application. React.js, a popular JavaScript library, was chosen for building the user interface due to its component-based structure, performance efficiency, and scalability. React enabled dynamic rendering of components such as patient search results, appointment modals, and live updates on medical records. Client-side validations and asynchronous operations were also managed via React hooks and lifecycle methods.

The frontend was built with a web-first approach to ensure accessibility for Doctors, Nurses, Administrators and Students accessing the system through browsers over HTTPS.

3.6.2 Backend Development

The backend development of the Healthcare Data Management System focused on managing the application's logic, handling data processing, securing medical records, and facilitating seamless communication between the user interface and the cloud infrastructure. It was responsible for executing core functionalities such as user authentication, patient registration, and appointment scheduling. The technologies employed for the backend include:

AWS Lambda: AWS Lambda served as the primary compute service for executing backend logic. It was chosen for its serverless, event-driven architecture, which enables the system to handle multiple requests concurrently without managing servers. In this system,

Lambda functions handled major operations such as processing patient data, updating prescriptions, and managing clinical logs.

Amazon API Gateway: Amazon API Gateway was used to develop the RESTful APIs that connect the frontend with the Lambda functions. These APIs handled various HTTP requests for user login, record updates, and scheduling. Its support allowed for effective request validation, throttling, and secure exposure of endpoints.

Amazon Cognito: For secure session management, Amazon Cognito was implemented to authenticate and authorize clinic staff. Upon successful login, secure tokens are issued and used to verify protected routes, ensuring that only authorized individuals like Doctors or Pharmacists can access sensitive medical information.

Amazon DynamoDB & S3: Amazon DynamoDB was used as the NoSQL database to store structured clinical data such as patient profiles and encounters. Amazon S3 was integrated to handle the storage of supporting documents and prescriptions. This structure allowed for real-time processing of records and high-performance data retrieval.

3.6.3 Development Tools

The development of the Healthcare Data Management System involved the use of several tools that facilitated efficient coding, testing, debugging, and cloud resource management. These tools supported both the frontend and backend development processes. The tools used are outlined below:

Visual Studio Code (VS Code): Visual Studio Code was the primary integrated development environment (IDE) used for writing and managing the project's source code.

Its lightweight structure and extensions for AWS and React.js made it suitable for the entire stack. Features such as IntelliSense and Git integration improved productivity and code quality.

Google Chrome: Google Chrome served as the main web browser for testing and rendering the user interface. Chrome's Developer Tools (DevTools) were particularly useful for inspecting HTML/CSS elements, debugging JavaScript, and analyzing network requests sent to the API Gateway.

Postman: Postman was utilized for testing the RESTful API endpoints developed via API Gateway and Lambda. It enabled the simulation of HTTP requests to verify that the backend responded correctly with the intended clinical data. Postman also helped in testing authentication using Cognito tokens.

AWS Management Console: The AWS Console provided a graphical interface for interacting with cloud services. It allowed for the easy visualization and monitoring of DynamoDB tables, S3 buckets, and Lambda logs. This tool was essential for checking data consistency and debugging issues related to cloud resource integration.

